

PRACTICAL-6

Performs finite-state analysis on a sentence to check its validity

Finite-State Automaton (FSA)

- An FSA is a mathematical model that recognizes whether a sequence of words/symbols follows certain grammar rules.
- It consists of states, transitions, and accepting states.
- In your second code:
 - States represent progress in parsing a sentence (e.g., start → determiner → adjective → noun → verb).
 - Transitions occur based on the category of the word (DET, ADJ, NOUN, VERB).
 - If the sentence ends in the accepting state (S4), it is valid; otherwise, it is rejected.

Example: "The big red cat jumped" → follows grammar path DET → ADJ → NOUN → VERB → valid.

CODE:-

```
# Define the states of the FSA
# S0: Start state
# S1: Found a determiner
# S2: Found an adjective
# S3: Found a noun
# S4: End state (found a verb, sentence is complete)

vocabulary = {
    'det': ['a', 'the'],
    'adj': ['big', 'red', 'tall'],
    'noun': ['cat', 'ball', 'man'],
    'verb': ['jumped', 'ran', 'threw']
}

transitions = {
    ('S0', 'det'): 'S1',
    ('S1', 'adj'): 'S2',
    ('S1', 'noun'): 'S3',
    ('S2', 'noun'): 'S3',
    ('S3', 'verb'): 'S4'
}

def analyze_sentence(sentence):

    words = sentence.lower().split()
    current_state = 'S0'
    path = [current_state]
```

```

print(f"Analyzing sentence: '{sentence}'")

for word in words:
    found_category = None

    for category, words_list in vocabulary.items():
        if word in words_list:
            found_category = category
            break

    if found_category is None:
        print(f"Error: Unknown word '{word}'.")
        return False

    transition = (current_state, found_category)
    if transition in transitions:
        current_state = transitions[transition]
        path.append(current_state)
        print(f" '{word}' ({found_category}) -> New state: {current_state}")
    else:
        print(f"Error: Invalid transition from state {current_state} with input '{word}' ({found_category}).")
        return False

if current_state == 'S4':
    print("\n Sentence is valid based on the FSA.")
    print("State Path:", ' -> '.join(path))
    return True

    -----
else:
    print("\n Sentence is incomplete or invalid.")
    return False

analyze_sentence("The big cat jumped")
print("-" * 30)
analyze_sentence("A man ran")
print("-" * 30)
analyze_sentence("The cat threw a ball")
print("-" * 30)
analyze_sentence("A ran")

```

Output:

Analyzing sentence: 'The big cat jumped'

'the' (det) -> New state: S1

'big' (adj) -> New state: S2

'cat' (noun) -> New state: S3

'jumped' (verb) -> New state: S4

Sentence is valid based on the FSA.

State Path: S0 -> S1 -> S2 -> S3 -> S4

Analyzing sentence: 'A man ran'

'a' (det) -> New state: S1

'man' (noun) -> New state: S3

'ran' (verb) -> New state: S4

Sentence is valid based on the FSA.

State Path: S0 -> S1 -> S3 -> S4

Analyzing sentence: 'The cat threw a ball'

'the' (det) -> New state: S1

'cat' (noun) -> New state: S3

'threw' (verb) -> New state: S4

Error: Invalid transition from state S4 with input 'a' (det).

Analyzing sentence: 'A ran'

'a' (det) -> New state: S1

Error: Invalid transition from state S1 with input 'ran' (verb).

Finite-State Transducer (FST):

- An FST extends the FSA: instead of just checking if input is valid, it also produces an output tape (a transformed version of the input).
- Used widely in lexical analysis, morphology, and NLP (e.g., converting surface words to lemmas + grammatical tags).
- In the first code I gave you:
 - Input = "The quick brown fox runs"
 - Output (lexical tape) = the+DET quick+ADJ brown+ADJ fox+N+SG run+V+3SG
- The FST both accepts the input and transduces it into structured information.

Example: "dogs" → dog+N+PL

CODE:-

```

▶ # Simple FST simulation

class FST:
    def __init__(self, lexicon):
        self.lexicon = lexicon

    def analyze(self, sentence):
        words = sentence.split()
        lexical_tape = []

        for word in words:
            if word.lower() in self.lexicon:
                lexical_tape.append(self.lexicon[word.lower()])
            else:
                lexical_tape.append(word + "+UNK")
        return " ".join(lexical_tape)

```

```

▶ # Example lexicon (surface -> lexical form)
lexicon = {
    "cats": "cat+N+PL",
    "cat": "cat+N+SG",
    "dogs": "dog+N+PL",
    "dog": "dog+N+SG",
    "run": "run+V+BASE",
    "runs": "run+V+3SG",
    "ran": "run+V+PAST",
    "the": "the+DET",
    "a": "a+DET",
    "quick": "quick+ADJ",
    "quickly": "quick+ADB",
    "brown": "brown+ADJ",
    "fox": "fox+N+SG",
}

# Create the FST;
fst = FST(lexicon)

sentence = "The cats ran quickly"
output = fst.analyze(sentence)

print("Input sentence: ", sentence)
print("Lexical tape: ", output)

```

```

➡ Input sentence: The cats ran quickly
  Lexical tape: the+DET cat+N+PL run+V+PAST quick+ADB

```